Федеральное государственное бюджетное образовательное учреждение высшего образования
«РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА И ГОСУДАРСТВЕННОЙ СЛУЖБЫ при ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»

СИБИРСКИЙ ИНСТИТУТ УПРАВЛЕНИЯ-филиал РАНХиГС

Направление подготовки: «Информационные системы и программирование»

Образовательная программа: 09.02.07

Проект

по дисциплине: Основы объектно-ориентированного программирования

на тему:

«Разработка мобильного приложения карточной игры «Гвинт» на платформе Android»

Автор работы:
Студент 4 курса группы К-ИСП-2302
Очной формы обучения
Васильев Константин Сергеевич

Новосибирск, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1. Описание бизнес-логики проекта 5
1.1 Назначение системы 5
1.2 Основные сценарии использования 5
1.3 Ключевые сущности и их поведение 7
1.4 Бизнес-правила 8
2. Тип архитектуры: монолит или микросервисы 10
2.1 Обоснование выбора архитектуры 10
2.2 Модули и слои 10
2.3 Преимущества и недостатки выбранного подхода 12
3. Хранение данных 13
3.1 Способ хранения данных 13
3.2 Хранимые сущности и атрибуты 13
3.3 Схема данных 14
3.4 Виды связей между сущностями 15
4. Проверка безопасности приложений 15
4.1 Актуальные угрозы 15
4.2 Меры защиты 16
4.3 Инкапсуляция и защита данных в проекте 17
5. Законы и нормативные акты, необходимые для внедрения информационной системы . 18
5.1 Применимые нормативные акты 18
5.2 Учёт требований при работе с данными 19
5.3 Согласие пользователей и политика конфиденциальности 20
6. Документы к каждому этапу процесса разработки 21
6.1 Анализ требований 21
6.2 Проектирование 21
6.3 Разработка 22
6.4 Тестирование 22
6.5 Внедрение 23
6.6 Сопровождение 24
6.7 Заключение 24
7. Индивидуальные акценты по ролям 25
7.1 Архитектор ядра (Backend / Core OOP Developer) 25
7.1.1 Иерархия классов 25
7.1.2 Принцип DRY 26

7.1.3 Описание бизнес-логики на уровне классов и методов 26
7.1.4 Примеры кода 28
7.1.5 UML-диаграмма классов 29
7.2 Роль: Разработчик интерфейса (Frondend / UI Intergration) 30
7.2.1 Организация интерфейса 30
7.2.2 Соблюдение инкапсуляции 31
7.2.3 Сценарии ввода и вывода данных 31
7.3 Роль: Специалист по безопасности и валидации данных (Security & Data Intergrity) 32
7.3.1 Приватные и защищённые атрибуты 32
7.3.2 Сеттеры и геттеры как фильтры 33
7.3.3 Проверки типов и значений 33
7.3.4 Исключения и обработка ошибок 34
7.3.5 Модуль валидации 35
7.3.6 Законы и персональных данных 36
7.4 Заключение 36

ВВЕДЕНИЕ

Целью проекта является разработка мобильного приложения «Гвинт», воспроизводящего оригинальную версию коллекционной карточной игры, с возможностью сбора колоды, проведения сражений и использования карт с различными способностями в соответствии с оригиналом.

Задачи проекта:

1. Выполнить проектирование архитектуры мобильного приложения;
2. Реализовать основные игровые механики: размещение карт на трёх рядах, учёт погодных эффектов, применение способностей карт;
3. Разработать искусственный интеллект противника, принимающий решения на основе приоритетов ходов;
4. Создать пользовательский интерфейс для сборки колоды, проведения сражения, просмотра карт и управления игровым процессом;
5. Провести тестирование разработанного приложения, выявить и устранить ошибки.

1. Описание бизнес-логики проекта

1.1 Назначение системы

Проект представляет собой мобильное приложение — коллекционную карточную игру «Гвинт» для платформы Android. Система воспроизводит оригинальную версию игры, изначально появившуюся как мини-игра внутри компьютерной ролевой игры. Бизнес-логика — совокупность правил, алгоритмов и сценариев, определяющих поведение системы, — построена вокруг игрового процесса «Гвинта». Основная задача приложения — предоставить пользователю возможность играть в классический «Гвинт» на мобильном устройстве без обязательного подключения к интернету.

Актуальность проекта обусловлена тем, что современные коммерческие версии игры претерпели значительные изменения по сравнению с оригиналом: были переработаны правила, добавлены микротранзакции, изменены способности многих карт, введено обязательное сетевое подключение. Оригинальная версия осталась недоступной на мобильных платформах, что создаёт устойчивый спрос среди поклонников классической механики.

1.2 Основные сценарии использования

Сценарий использования — описание типового взаимодействия пользователя с системой, включающее последовательность действий и ожидаемый результат. Система поддерживает следующие сценарии:

— Сборка колоды — игрок открывает редактор колоды, добавляет и удаляет карты, формирует собственную стратегию.

— Сортировка карт по фракциям — игрок фильтрует коллекцию по принадлежности к фракции, что ускоряет поиск нужных карт.

— Выбор лидера — игрок выбирает карту лидера, обладающую уникальной способностью.

— Начало сражения — игрок запускает битву из главного меню с сохранённой колодой и лидером.

— Муллиган — в начале партии игрок заменяет до трёх карт в стартовой руке.

— Розыгрыш карты — игрок кликает по карте в руке, она анимировано перемещается на соответствующий ряд.

— Выбор ряда — для карт с нефиксированным расположением игрок выбирает ряд вручную.

— Использование способности карты — при выкладывании карты срабатывает её особая способность.

— Активация способности лидера — один раз за партию игрок активирует способность лидера.

— Пас — игрок отказывается от дальнейших ходов в текущем раунде.

— Просмотр карты долгое нажатие открывает диалог с увеличенным изображением и описанием способностей.

— Просмотр сброса игрок открывает свой сброс или сброс противника.

— Настройка звука регулировка громкости музыки и звуковых эффектов.

В формате пользовательских историй ключевые требования формулируются так: «Как игрок, я хочу собирать колоду из доступных карт, чтобы подготовить стратегию перед боем»; «Как игрок, я хочу активировать способность лидера один раз за партию, чтобы получить тактическое преимущество»; «Как игрок, я хочу заменить до трёх карт в стартовой руке, чтобы улучшить начальную руку».

Визуальную иллюстрацию вариантов использования можно увидеть на рисунке 1.

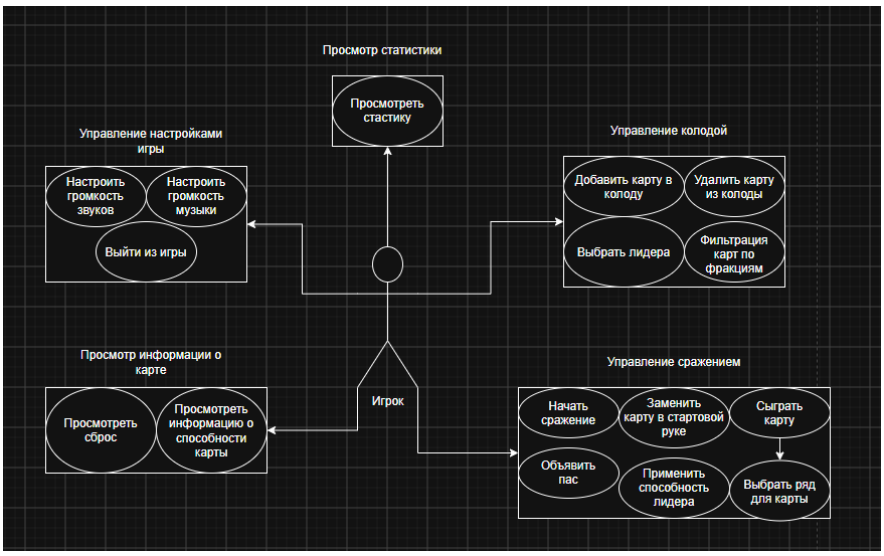


Рисунок 1. Диаграмма вариантов использования мобильного приложения «Гвинт»

1.3 Ключевые сущности и их поведение

Сущность — отдельный объект предметной области, обладающий набором атрибутов и определённым поведением. Бизнес-логика проекта построена вокруг следующих сущностей:

— Карта (Cards) — описывает игровую карту: изображение, название, силу, ряд, способность и фракцию.

— Состояние боя (BattleState) хранит данные о рядах обоих игроков, руках, колодах, сбросах, погодных эффектах и счёте.

— Экран сражения (BattleActivity) управляет ходом партии, обработкой действий игрока и работой искусственного интеллекта.

— Редактор колоды (DeckActivity) — отвечает за сборку колоды и выбор лидера.

— Главное меню (MainActivity) — точка входа в приложение.

— Справочник карт (CardsData) содержит данные обо всех картах, лидерах, группах связки и фракциях.

— Менеджер способностей (AbilityManager) обрабатывает применение способностей карт в зависимости от их типа.

— Калькулятор очков (ScoreCalculator) вычисляет силу карт и суммарный счёт с учётом погоды, горна и связки.

— Менеджер погоды (WeatherManager) управляет погодными эффектами и их визуальным отображением.

— Менеджер сброса (GraveyardManager) отвечает за перемещение карт в сброс и воскрешение.

— Искусственный интеллект (EnemyAI) принимает решения на основе системы приоритетов.

— Менеджер бонусов фракций (FactionAbilityManager) применяет способности фракций между раундами.

— Репозиторий колоды (DeckRepository) обеспечивает сохранение и загрузку колоды и лидера.

— Менеджер звука (SoundManager) — управляет фоновой музыкой и звуковыми эффектами.

— Менеджер сообщений (MessageManager) отображает всплывающие уведомления в ходе боя.

Каждая сущность отвечает строго за свою область и взаимодействует с другими через публичные методы, что обеспечивает контроль над изменениями состояния.

1.4 Бизнес-правила

Игровой процесс регулируется следующими правилами:

— Партия ведётся до двух побед в раундах.

— Раунд завершается, если оба игрока объявили пас или у одного из них закончились карты на руке.

— Право первого хода в раунде определяется случайным выбором, за исключением, если у игрока фракция «Скоя'таэли» — первым ходит игрок, если у противника — противник; если фракция «Скоя'таэли» есть у обоих — первый ход определяется случайным выбором.

— Победитель предыдущего раунда получает фракционный бонус (например, добор карты для Северных королевств).

— Карты-герои не подвержены воздействию погодных эффектов и способностей других карт.

— Карты погоды снижают силу обычных карт в соответствующих рядах до единицы: «Мороз» действует на ближний ряд, «Мгла» — на дальний, «Ливень» — на осадный.

— Карта «Ясное небо» снимает все активные погодные эффекты.

— Карта «Горн» удваивает силу обычных карт в ряду.

— Способность «Связка» умножает силу карт одной группы на их количество в ряду.

— Способность «Шпион» размещает карту на стороне противника, взамен позволяя игроку взять две карты из колоды.

— Способность «Медик» воскрешает одну карту из кладбища.

— Способность «Чучело» возвращает свою не-герой карту с поля в руку.

— Способность «Казнь» (execution) удаляет с поля карты противника с максимальной силой в указанных рядах (кроме карт-героев).

— Способность лидера доступна один раз за партию.

— В начале партии игрок может заменить до трёх карт в стартовой руке.

Все перечисленные правила формализуются в игровой логике, проверяются в ходе тестирования и фиксируются в сопроводительной документации.

2. Тип архитектуры: монолит или микросервисы

2.1 Обоснование выбора архитектуры

Архитектура программного обеспечения — это совокупность решений о структуре системы, определяющих её компоненты, их взаимодействие и правила организации. Существует два основных подхода: монолит, при котором приложение представляет собой единый развёртываемый модуль, и микросервисы, при котором система разделена на независимо развёртываемые сервисы, взаимодействующие по сети.

Для разрабатываемого проекта была выбрана монолитная архитектура. Это обосновано спецификой приложения: оно представляет собой автономное мобильное приложение для платформы Android, работающее без серверной части. Микросервисная архитектура предполагает разделение системы на независимые сервисы, взаимодействующие по сети, что для офлайн-игры на одном устройстве избыточно. Отсутствует сетевое взаимодействие, нет необходимости в масштабировании отдельных компонентов и в распределённых транзакциях.

2.2 Модули и слои

Внутри монолита выделены слои, обеспечивающие разделение ответственности — принцип, при котором каждая часть системы отвечает строго за свою область и не вмешивается в работу других.

Слой пользовательского интерфейса отвечает за отображение игрового процесса и обработку действий пользователя. В него входят экраны главного меню (MainActivity), выбора режима (ModeSelectActivity), редактора колоды (DeckActivity) и сражения (BattleActivity), а также диалог предпросмотра карт (CardPreviewDialogFragment) и адаптер для их отображения (CardsAdapter). За реализацию этого слоя отвечает разработчик интерфейса.

Слой игровой логики содержит правила и алгоритмы игры. В него входят состояние боя (BattleState), подсчёт очков (ScoreCalculator), управление погодными эффектами (WeatherManager), кладбищами (GraveyardManager),

способностями карт (AbilityManager), бонусами фракций (FactionAbilityManager) и искусственный интеллект (EnemyAI). Этот слой проектируется и реализуется архитектором ядра.

Слой данных отвечает за модель карты (Cards), справочник всех карт, лидеров и фракций (CardsData), а также за сохранение и загрузку колоды и лидера (DeckRepository). Дополнительно в этот слой входит менеджер статистики (StatsManager), сохраняющий счётчики побед, поражений и общего числа матчей. За корректность работы этого слоя отвечает специалист по безопасности и валидации данных.

Слой утилит содержит вспомогательные компоненты: менеджер звука (SoundManager), менеджер сообщений (MessageManager) и функции-расширения для работы со списками карт (CardExtensions).

Взаимодействие слоёв построено по принципу «сверху вниз»: пользовательский интерфейс вызывает методы игровой логики, игровая логика обращается к данным, а утилиты используются всеми слоями по мере необходимости. Такая организация исключает циклические зависимости и упрощает поддержку кода.

Схема взаимодействия компонентов представлена на рисунке 2.

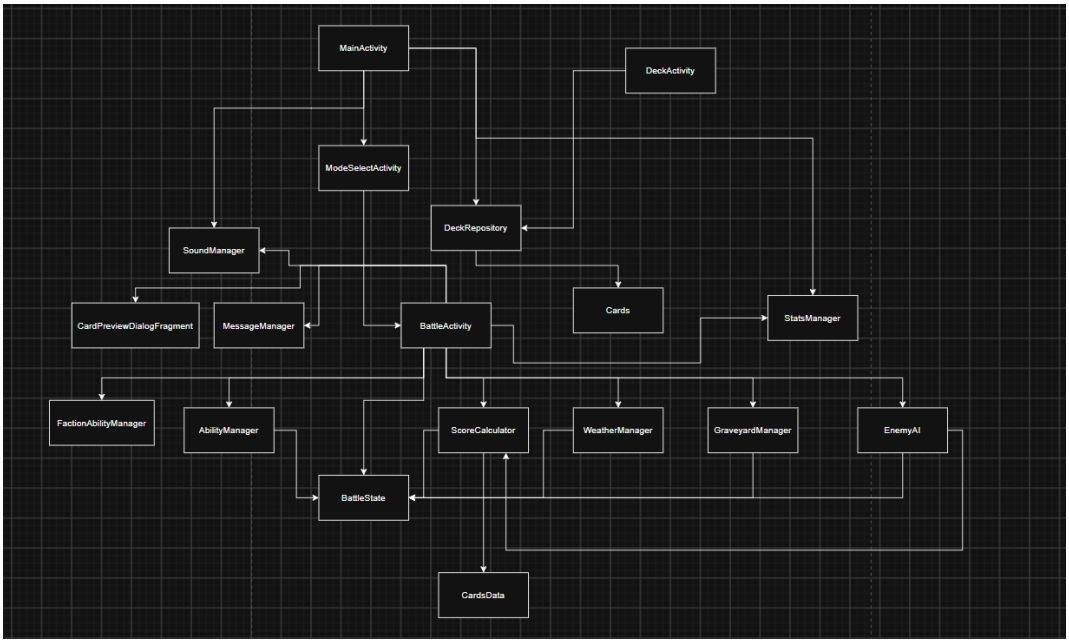


Рисунок 2. Диаграмма взаимодействия компонентов мобильного приложения «Гвинт»

2.3 Преимущества и недостатки выбранного подхода

Преимущества монолитной архитектуры для проекта:

— Простота развёртывания — приложение собирается в один APK-файл и устанавливается на устройство как единое целое.

— Отсутствие сетевых задержек — все вызовы между слоями происходят внутри одного процесса, что обеспечивает мгновенный отклик.

— Лёгкость отладки — все компоненты работают в едином процессе, что упрощает поиск ошибок.

— Отсутствие серверной инфраструктуры — приложение не требует хостинга, баз данных на сервере и сетевого взаимодействия.

— Недостатки монолитной архитектуры:

— Отсутствие жёсткой изоляции модулей — изменения в одном слое могут косвенно затронуть другой.

— Потенциальные сложности при масштабировании — если проект значительно вырастет, поддержка монолита усложнится.

— Единый артефакт — обновление любой части требует пересборки и повторной установки всего приложения.

Для текущих задач и планируемого расширения функциональности монолитный подход является оптимальным выбором. Он обеспечивает быструю разработку, простоту поддержки и отсутствие избыточной инфраструктуры.

3. Хранение данных

3.1 Способ хранения данных

Хранение данных — это способ организации долговременного сохранения информации, позволяющий приложению восстанавливать состояние между запусками. В мобильной разработке применяются реляционные базы данных, файловые хранилища, встроенные механизмы платформы и облачные сервисы.

В разрабатываемом проекте используется локальное хранение данных. Приложение работает автономно и не передаёт данные на серверы, поэтому необходимости в СУБД или облачных решениях нет. Для хранения применяются два механизма платформы Android:

SharedPreferences — встроенное хранилище типа «ключ-значение», предназначенное для небольших наборов параметров. В проекте в нём сохраняются настройки громкости музыки и звуковых эффектов. Третья группа SharedPreferences — «stats» — используется классом StatsManager для хранения агрегированной статистики матчей. Хранятся три целочисленных счётчика: games (всего матчей), wins (побед), losses (поражений). Запись выполняется методами addWin() и addLoss(), чтение — getGames(), getWins(), getLosses(). Эти данные используются в главном меню (MainActivity.updateStatsText()) и обновляются по завершении партии (BattleActivity.showEndGameScreen()).

JSON-сериализация — формат представления структурированных данных в виде текста. Используется для сохранения списка карт колоды и выбранного лидера. Сериализация выполняется с помощью библиотеки Gson, преобразующей объекты в JSON-строку и обратно. Полученная строка сохраняется в том же SharedPreferences.

Дополнительно используются ресурсы приложения для хранения статических данных: описания способностей карт в текстовом файле, а также исходный код в качестве справочника всех карт, лидеров и групп связки.

3.2 Хранимые сущности и атрибуты

Перечень хранимых сущностей и их атрибутов:

Карта — основная игровая единица. Атрибуты: изображение, название, сила, ряд, способность, видимость в сбросе, фракция.

Лидер — особая карта с уникальной способностью. Атрибуты: изображение, название, способность, фракция.

Колода — список карт, выбранных игроком для сражения.

Настройки — пользовательские параметры. Атрибуты: громкость музыки, громкость звуковых эффектов.

Статистика — агрегированные счётчики матчей. Атрибуты: games, wins, losses. Хранится в SharedPreferences «stats», доступ через StatsManager.

Описания способностей — текстовые пояснения к каждой способности карт.

3.3 Схема данных

Логическая схема хранимых данных, отражающая сущности и связи между ними, приведена на рисунке 3. Физически данные размещены в трёх группах SharedPreferences: «deck» (колода и лидер в JSON), «settings» (громкость музыки и звуков), «stats» (счётчики игр/побед/поражений).

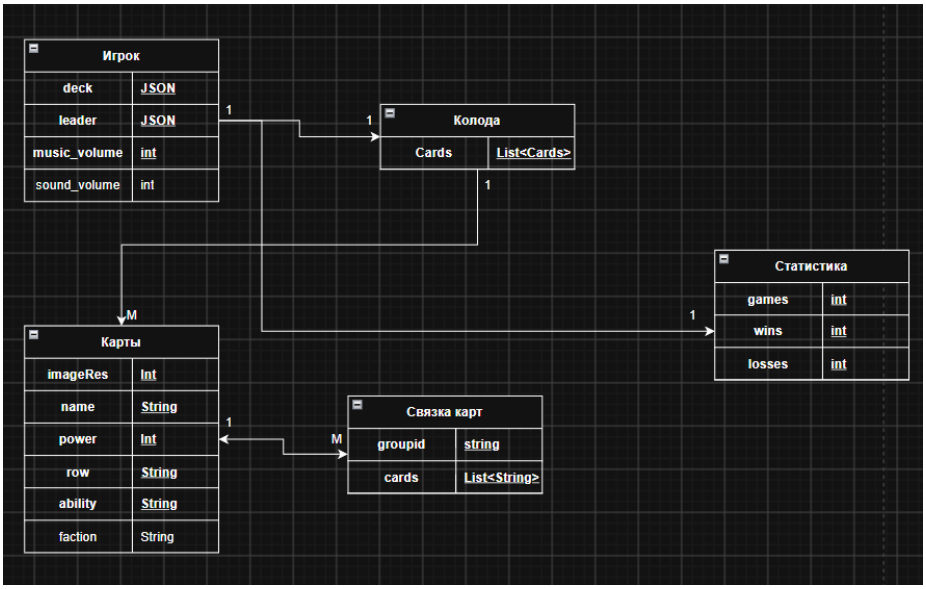


Рисунок 3. Схема данных мобильного приложения «Гвинт»

3.4 Виды связей между сущностями

Поскольку используется файловое хранилище, классические реляционные связи не применяются напрямую. Однако в качестве примера можно выделить три типа связей из данного проекта:

Связь «один-к-одному» — это отношение, при котором одной записи в первой сущности соответствует ровно одна запись во второй. В проекте такая связь проявляется в отношении игрока к выбранному лидеру: у игрока может быть только один лидер.

Связь «один-ко-многим» — это отношение, при котором одной записи в первой сущности соответствует несколько записей во второй. В проекте такая связь проявляется в отношении колоды к картам: одна колода содержит множество карт.

Связь «многие-ко-многим» — это отношение, при котором нескольким записям одной сущности соответствует несколько записей другой. В текущей версии проекта такая связь в явном виде не используется: справочники bondGroups и musterGroups реализованы как Map<String, List<String>>, где каждая карта однозначно определяется по имени и относится ровно к одной группе (getBondGroup / getMusterGroup возвращают единственное значение или null). Поэтому отношение «карта — группа» фактически является связью «один-ко-многим»: одна группа содержит несколько карт, но каждая карта входит только в одну группу. Связь M:N в проекте не применяется и приведена в тексте как теоретический случай для полноты классификации.

4. Проверка безопасности приложений

4.1 Актуальные угрозы

Информационная безопасность приложения — это состояние защищённости данных и логики системы от несанкционированного доступа, случайного повреждения и вредоносного воздействия. При анализе угроз важно учитывать специфику конкретного приложения, а не рассматривать универсальный список.

Для разрабатываемого проекта характерны следующие особенности:

— приложение работает локально, без сетевого взаимодействия;

— не использует базы данных SQL;

— не имеет веб-интерфейса;

— не собирает персональные данные пользователей.

Исходя из этого, часть типовых угроз неактуальна для проекта.

SQL-инъекции — это атаки, при которых в SQL-запрос внедряется вредоносный код. Не актуально, поскольку SQL не используется.

XSS — это межсайтовый скриптинг, при котором в веб-страницу внедряется вредоносный JavaScript. Не актуально, поскольку веб-интерфейса нет.

CSRF — это подделка межсайтовых запросов, при которой злоумышленник заставляет пользователя выполнить нежелательное действие. Не актуально, поскольку сетевые запросы отсутствуют.

Слабая аутентификация — не актуальна, поскольку учётные записи в приложении отсутствуют.

Актуальными для проекта остаются следующие три угрозы:

1. утечка данных при потере устройства — колода и настройки хранятся локально и доступны при физическом доступе к устройству;

2. повреждение сохранений — случайное или намеренное редактирование файлов хранилища может нарушить целостность данных;

3. некорректные данные в справочнике карт — ошибки в описаниях карт и способностей могут нарушить игровую логику.

4.2 Меры защиты

Для перечисленных угроз применяются следующие меры защиты.

Валидация данных — проверка корректности входных значений перед их использованием. При загрузке сохранений реализована проверка наличия ключа в SharedPreferences: если колода или лидер ранее не сохранялись, используются

значения по умолчанию, что исключает передачу пустых или неинициализированных данных в игровую логику.

Ограничение диапазона значений — числовые параметры, такие как громкость музыки и звука, ограничиваются допустимым диапазоном от 0 до 100. Это исключает выход за пределы корректных значений.

Обработка исключений — критичные операции чтения файлов и ресурсов оборачиваются в конструкцию try-catch, предотвращающую аварийное завершение приложения при ошибках.

Доступ к хранилищу SharedPreferences инкапсулирован в отдельном классе-репозитории, что исключает прямое взаимодействие других слоёв приложения с хранилищем.

Ограничение игровых действий — количество замен карт в стартовой руке ограничено тремя, что соответствует правилам игры и исключает возможность некорректного использования механики муллигана.

Хеширование паролей — не применяется, поскольку в приложении отсутствует аутентификация.

HTTPS — не применяется, поскольку приложение не взаимодействует с сетью.

Роли и права доступа — не применяются, поскольку приложение предназначено для одного пользователя.

Логирование — в проекте применяется ограниченно: события записываются средствами отладки в процессе разработки, но постоянное логирование в релизной версии не ведётся.

4.3 Инкапсуляция и защита данных в проекте

Инкапсуляция — принцип объектно-ориентированного программирования, при котором внутренние данные объекта скрыты от внешнего доступа и изменяются только через публичные методы.

В проекте реализованы следующие механизмы инкапсуляции и защиты данных.

Приватные атрибуты. Большинство полей состояния и менеджеров объявлены как приватные и недоступны для изменения извне. Это исключает случайное или намеренное изменение состояния в обход методов класса.

Публичные методы как контролируемые точки входа. Изменение состояния боя происходит только через публичные методы менеджеров: применение способностей, розыгрыш карт, смена хода. Это позволяет централизованно контролировать все изменения и отслеживать их источник.

Модель карты как неизменяемая структура. Большая часть атрибутов карты — такие как изображение, название, ряд, способность, фракция — объявлены как неизменяемые. Изменяемыми остаются только те поля, которые действительно изменяются в ходе игры: сила карты и признак видимости в сбросе.

Проверки типов и значений. Все числовые параметры, получаемые из внешних источников, приводятся к допустимому диапазону. Строковые параметры проверяются на соответствие ожидаемым значениям.

Изоляция хранилища. Прямой доступ к SharedPreferences осуществляется только из класса-репозитория. Остальные слои приложения работают с сохранёнными данными через методы репозитория, что исключает случайное повреждение данных.

Такая организация обеспечивает устойчивость приложения к некорректным данным и снижает риск случайного нарушения игровой логики.

5. Законы и нормативные акты, необходимые для внедрения информационной системы

5.1 Применимые нормативные акты

Информационная система — это совокупность программных и аппаратных средств, предназначенных для сбора, хранения, обработки и выдачи информации. Внедрение любой информационной системы регулируется законодательством в области информации и защиты данных. Для разрабатываемого проекта применимы следующие нормативные акты.

Федеральный закон № 152-ФЗ «О персональных данных» — регулирует отношения, связанные с обработкой персональных данных, то есть любых сведений, относящихся к конкретному человеку. Закон устанавливает требования к сбору, хранению, обработке и защите таких данных. Для разрабатываемого приложения требования этого закона минимальны, поскольку система не собирает и не обрабатывает персональные данные пользователей.

Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и защите информации» — устанавливает общие принципы работы с информацией, определяет правовой режим её распространения, а также регулирует отношения между участниками информационного обмена. Закон применим к проекту в общем виде как к любому программному продукту, работающему с информацией.

Регламент GDPR — общий регламент по защите данных, действующий на территории Европейского союза. Для проекта неактуален, поскольку приложение ориентировано на российский рынок и не распространяется среди пользователей ЕС.

Локальные политики компании — внутренние документы организации, регулирующие использование программного обеспечения. Применимы, если приложение будет использоваться в корпоративной среде.

5.2 Учёт требований при работе с данными

При сборе, хранении и обработке данных в проекте соблюдаются следующие принципы.

Минимизация данных — принцип, при котором собираются только те данные, которые необходимы для работы системы. Приложение хранит колоду игрока, пользовательские настройки громкости и агрегированную статистику матчей (всего игр, побед, поражений). Персональные сведения — имя, контакты, местоположение — не собираются.

Локальное хранение — все данные сохраняются исключительно на устройстве пользователя и не передаются на сторонние серверы. Это исключает необходимость выполнения требований к защите данных при передаче по сети.

Отсутствие трансграничной передачи — данные не покидают устройство, что снимает вопросы о соответствии требованиям законодательства других стран.

Прозрачность хранения — пользователь может самостоятельно удалить все данные, удалив приложение. Политика конфиденциальности при этом не требуется, поскольку персональные данные не обрабатываются.

5.3 Согласие пользователей и политика конфиденциальности

Согласие пользователя — это добровольное разрешение на обработку персональных данных. Поскольку приложение не собирает персональные данные, получение согласия не требуется.

Политика конфиденциальности — документ, описывающий, какие данные собирает приложение и как они используются. В текущей версии проекта политика конфиденциальности не разрабатывается, так как приложение не собирает и не передаёт данные.

Однако при публикации приложения в магазинах Google Play или RuStore потребуется соблюдение правил этих площадок. Правила требуют предоставления декларации о работе с данными даже для приложений, которые их не собирают. Поэтому перед возможной публикацией потребуется подготовить декларацию, подтверждающую отсутствие сбора персональных данных.

6. Документы к каждому этапу процесса разработки

6.1 Анализ требований

Анализ требований — это этап разработки, на котором формулируются цели системы, её функции и ограничения. От качества этого этапа зависит, насколько точно конечный продукт будет соответствовать ожиданиям.

На этом этапе формируются следующие документы.

Техническое задание — основной документ, описывающий назначение системы, требования к функциональности, интерфейсу и качеству. Определяет, что именно должна делать система и в каких условиях работать.

Список требований — перечень конкретных функциональных и нефункциональных требований, представленных в виде пронумерованных пунктов. Используется для контроля полноты реализации.

Пользовательские истории — краткие описания сценариев использования системы от лица пользователя. Формулируются в виде «как пользователь, я хочу выполнить действие, чтобы получить результат». Применяются для приоритизации разработки.

После утверждения требований проект переходит к этапу проектирования.

6.2 Проектирование

Проектирование — это этап, на котором определяются структура системы, её компоненты и их взаимодействие. На основе требований формируется архитектурное решение и детальная модель будущего приложения.

На этом этапе создаются следующие документы.

UML-диаграмма классов — схема, отображающая классы системы и связи между ними. Показывает, из каких сущностей состоит приложение и как они взаимодействуют друг с другом.

UML-диаграмма последовательностей — схема, показывающая порядок вызовов методов между объектами при выполнении конкретного сценария. Используется для документирования сложных сценариев, например розыгрыша карты или применения способности.

ER-диаграмма — схема, описывающая структуру хранения данных и связи между сущностями. Применяется для документирования формата хранения.

Схема архитектуры — общее представление о слоях и модулях приложения, их назначении и взаимодействии.

После утверждения проектной документации начинается разработка.

6.3 Разработка

Разработка — это этап реализации системы в соответствии с проектной документацией. На этом этапе создаётся исходный код приложения и ведётся рабочая документация.

На этом этапе формируются следующие документы.

Описание модулей — документ, в котором перечислены модули приложения и указано их назначение. Включает краткое описание ответственности каждого модуля и его места в архитектуре.

Структура кода — описание организации исходного кода: какие папки и пакеты используются, какие файлы за что отвечают, как группируются классы.

README — основной файл документации проекта, размещаемый в корне репозитория. Содержит краткое описание приложения, инструкцию по сборке и запуску, примеры использования и перечень зависимостей.

API — программный интерфейс взаимодействия между модулями. В проекте представлен набором публичных методов классов игровой логики, через которые осуществляется доступ к игровому состоянию.

После завершения разработки функциональности начинается тестирование.

6.4 Тестирование

Тестирование — это этап проверки соответствия системы заявленным требованиям. Цель этапа — выявить и устранить дефекты до передачи приложения пользователям.

На этом этапе формируются следующие документы.

Тест-план — документ, описывающий, какие модули и методы подлежат тестированию, какие виды тестов применяются и какие критерии приёмки используются.

Чек-листы — структурированные списки проверок, выполняемых при тестировании. Используются для ручного тестирования и проверки типовых сценариев.

Отчёты о дефектах — документы, описывающие выявленные ошибки: условия воспроизведения, ожидаемое и фактическое поведение, критичность. Применяются для отслеживания процесса исправлений.

Автотесты — программные тесты, выполняемые без участия человека. Применяются для проверки игровой логики и предотвращения регрессий при доработках.

После успешного прохождения тестирования приложение переходит к внедрению.

6.5 Внедрение

Внедрение — это этап передачи системы пользователям и её развёртывания. На этом этапе готовится сопроводительная документация для конечного пользователя.

На этом этапе формируются следующие документы.

Инструкция по развёртыванию — документ, описывающий процесс установки приложения на устройство. Включает требования к среде, порядок сборки APK и способы установки.

Руководство пользователя — документ, описывающий основные сценарии работы с приложением. Содержит пошаговые инструкции по сборке колоды, началу сражения и настройке приложения.

План миграции — документ, описывающий перенос данных при обновлении версии. В текущей архитектуре формат сохранений (JSON в SharedPreferences) является стабильным и не требует миграции между версиями.

После выпуска приложения начинается этап сопровождения.

6.6 Сопровождение

Сопровождение — это этап поддержки системы после её выпуска. Включает выпуск обновлений, исправление ошибок и обеспечение сохранности данных пользователей.

На этом этапе формируются следующие документы.

Регламент обновлений — документ, описывающий порядок выпуска новых версий: какие изменения вносятся, как проводится тестирование, какие каналы распространения используются.

Резервное копирование — процедура сохранения данных для предотвращения их утраты. В проекте данные хранятся локально в SharedPreferences и могут быть включены в системное резервное копирование Android (Auto Backup) в соответствии с настройками платформы.

Мониторинг — процедура отслеживания состояния системы в процессе эксплуатации. В проекте используется класс DebugLogger для записи ключевых событий в процессе разработки, а также стандартный вывод стеков исключений. Внедрение внешних систем мониторинга (например, Crashlytics) предусмотрено на этапе промышленной эксплуатации.

6.7 Заключение

Все перечисленные документы ведутся в едином репозитории проекта. Это обеспечивает централизованное хранение информации, удобный доступ к ней для всех участников команды и возможность отслеживания изменений. Документация обновляется по мере развития проекта и служит основой для передачи знаний между участниками и для дальнейшего сопровождения приложения.

7. Индивидуальные акценты по ролям

Данная глава посвящена индивидуальному вкладу автора в разработку проекта. Автор совмещает три роли: архитектора ядра, разработчика интерфейса и специалиста по безопасности и валидации данных. Поскольку архитектор ядра является основной ролью, именно её раскрытие занимает в главе центральное место.

7.1 Архитектор ядра (Backend / Core OOP Developer)

7.1.1 Иерархия классов

Игровое ядро проекта построено на принципах объектно-ориентированного программирования — подхода, при котором программа представляется как совокупность взаимодействующих объектов, каждый из которых обладает своим состоянием и поведением. В основе архитектуры лежат три механизма: наследование, композиция и полиморфизм.

Наследование — это механизм, позволяющий создавать новые классы на основе существующих, перенимая их свойства и поведение. В проекте наследование применяется на уровне активностей. Базовым классом для всех экранов является AppCompatActivity, предоставляющий стандартный жизненный цикл и инфраструктуру Android.

От базового класса наследуются методы жизненного цикла (onCreate, onPause, onResume, onDestroy), работа с контекстом приложения, доступ к ресурсам и менеджеру фрагментов. Дочерние классы переопределяют часть этих методов для добавления собственной логики: например, BattleActivity использует onCreate для инициализации игрового поля, а onDestroy — для освобождения ресурсов медиаплеера.

Композиция — это механизм, при котором одни объекты содержат в себе другие и делегируют им часть работы. В проекте композиция является основным способом организации кода. Экран сражения BattleActivity содержит менеджеры

ScoreCalculator, WeatherManager, GraveyardManager и EnemyAI, каждый из которых отвечает за свою область.

Полиморфизм в проекте реализуется через интерфейс BattleInterface. Он определяет контракт взаимодействия между игровым ядром и экраном сражения. Класс BattleActivity реализует этот интерфейс, а менеджер способностей (AbilityManager) работает не с конкретной активностью, а с интерфейсом, что позволяет внедрять любую другую реализацию без изменения логики ядра.

7.1.2 Принцип DRY

DRY (Don't Repeat Yourself) — принцип разработки, согласно которому повторяющиеся фрагменты кода должны быть вынесены в отдельные сущности. В проекте этот принцип реализован следующими способами.

Общие операции над списками карт вынесены в отдельный файл CardExtensions. Это функции-расширения Kotlin, позволяющие вызывать методы у списков так, как будто они встроены в язык. Например, функция withoutSpies() возвращает список карт без шпионов и используется как в менеджере кладбища, так и в логике добора карт.

Работа с сохранениями централизована в классе DeckRepository. Все операции чтения и записи колоды и лидера проходят через этот класс, что исключает дублирование кода работы с SharedPreferences.

Логика подсчёта очков вынесена в ScoreCalculator, который используется как экраном сражения, так и искусственным интеллектом. Это исключает дублирование формул подсчёта силы карт.

Обработка способностей карт сосредоточена в единственном классе AbilityManager. Каждая способность имеет собственный приватный метод, а общая маршрутизация выполняется через единую точку входа.

7.1.3 Описание бизнес-логики на уровне классов и методов

Игровая логика распределена между несколькими классами в соответствии с зонами ответственности.

BattleState — класс состояния боя. Хранит данные о рядах карт игрока и противника, руках, колодах, кладбищах, погодных эффектах, счёте раундов и флагах хода. Логики не содержит — только поля.

ScoreCalculator — класс подсчёта очков. Основные методы: calculateTotalPlayerScore() и calculateTotalEnemyScore() — вычисляют суммарный счёт игрока и противника; calculateRowScoreWithBonds() — подсчитывает силу ряда с учётом связки; calculateSingleCardPower() — вычисляет силу отдельной карты с учётом погоды, горна и других модификаторов.

WeatherManager — класс управления погодой. Основные методы: activateFrost(), activateFog(), activateRain() — активируют соответствующую погоду; clearWeather() — снимает все погодные эффекты; hideAllEffects() — скрывает визуальные эффекты при очистке поля.

GraveyardManager — класс управления сбросом. Основные методы: moveCardsToGraveyard() — перемещает карты из всех рядов в сбросе; getRemainingDeckSize() — подсчитывает оставшиеся в колоде карты; findInGraveyard() — ищет карту в сбросе для воскрешения; addToSpyGraveyard() — добавляет шпиона в скрытый сброс.

EnemyAI — класс искусственного интеллекта. Основные методы: shouldPass() — решает, объявлять ли пас; selectMove() — выбирает лучший ход на основе системы приоритетов; getCardBaseValue() — вычисляет базовую ценность карты; getAllPossibleMoves() — формирует список всех возможных ходов.

AbilityManager — класс применения способностей карт. Основной метод applyAbility() определяет тип способности карты и вызывает соответствующий приватный метод: applySpyAbility(), applyMedicAbility(), applyX2BondAbility(), applyDecoyAbility(), applyFrostAbility(), applyFogAbility(), applyRainAbility(), applyClearWeatherAbility(). Помимо базовых способностей, AbilityManager обрабатывает механики execution (уничтожение карт противника с наибольшей силой в заданных рядах) и особые случаи для карт по имени.

FactionAbilityManager — класс бонусов фракций. Основной метод applyFactionBonus() применяет способность фракции между раундами: для Северных королевств — дополнительный добор карты, для Монстров — сохранение одной карты на поле.

Такое разделение позволяет независимо изменять каждую часть логики, не затрагивая остальные.

7.1.4 Примеры кода

Пример полиморфного поведения через интерфейс представлен на рисунке 4.

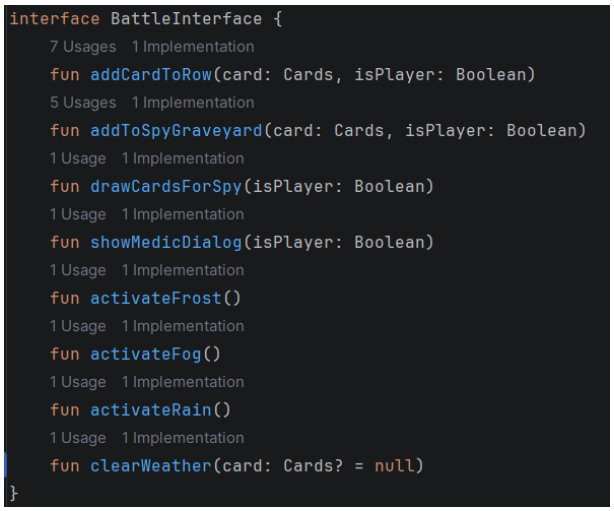


Рисунок 4. Пример кода полиморфмного поведения

Класс BattleActivity реализует этот интерфейс, предоставляя конкретную реализацию методов. Менеджер способностей AbilityManager работает не с конкретным классом активности, а с интерфейсом, что позволяет использовать его в других контекстах без переписывания.

Пример композиции представлен на рисунке 5.

```
private val scoreCalculator by lazy { ScoreCalculator(state) }
5 Usages
private val weatherManager by lazy {...}
6 Usages
private val graveyardManager by lazy { GraveyardManager(state) }
2 Usages
private val enemyAI by lazy { EnemyAI(state, scoreCalculator) }
```

Рисунок 5. Пример кода композиции

Экран сражения не содержит игровой логики напрямую — он делегирует её менеджерам, каждый из которых получает необходимые данные через общее состояние.

Пример принципа DRY можно увидеть на рисунке 6.

```
10 Usages
fun List<Cards>.withoutSpies(): List<Cards> = filter { it.ability != "spy" }
```

Рисунок 6. Пример применения принципа DRY

Одна функция используется во многих местах, что исключает дублирование.

7.1.5 UML-диаграмма классов

UML-диаграмма классов отображает структуру игрового ядра: базовые и дочерние классы, менеджеры, модели и связи между ними. Для наглядности на диаграмме представлены ключевые классы игровой логики; вспомогательные компоненты (менеджер фракций FactionAbilityManager, менеджер статистики StatsManager, экран выбора режима ModeSelectActivity) вынесены в другие разделы документации (Глава 2.2).

Диаграмма классов представлена на рисунке 7.

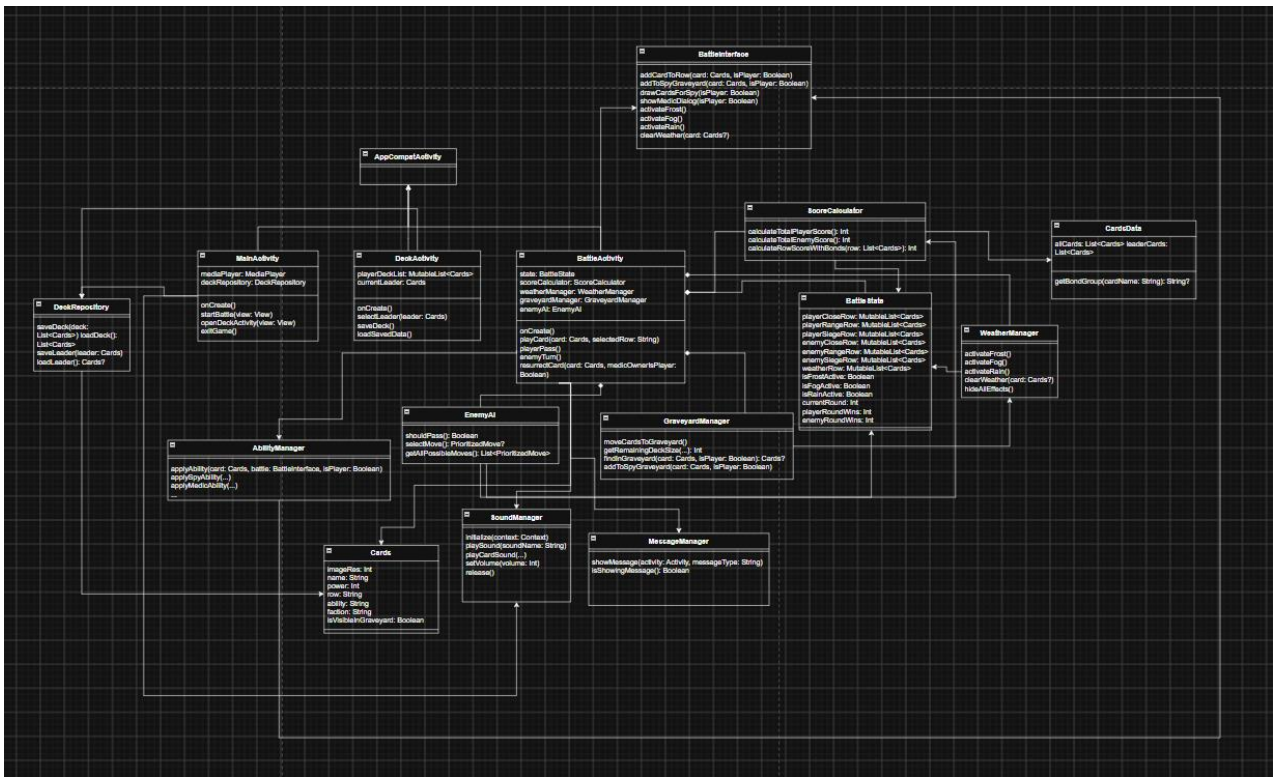


Рисунок 7. Диаграмма классов мобильного приложения «Гвинт»

7.2 Роль: Разработчик интерфейса (Frondend / UI Intergration)

7.2.1 Организация интерфейса

Интерфейс приложения построен на XML-разметке — декларативном способе описания внешнего вида экранов, при котором структура элементов задаётся отдельно от логики. Такой подход обеспечивает разделение кода разметки и кода поведения.

Каждый экран приложения реализован в отдельной активности. Главное меню — MainActivity, редактор колоды — DeckActivity, экран сражения — BattleActivity. Для отображения списков карт применяется компонент RecyclerView в сочетании с кастомным адаптером CardsAdapter.

RecyclerView — это компонент Android, предназначенный для эффективного отображения больших списков. Он переиспользует элементы, выходящие за пределы экрана, что снижает нагрузку на память.

Адаптер — это класс, который связывает данные со списком и создаёт элементы интерфейса по мере необходимости.

View-классы и обёртки. В проекте интерфейс построен на стандартных View-элементах Android (RecyclerView, ImageView, TextView, Button, SeekBar), описанных в XML-разметке. Роль «обёрток», инкапсулирующих отображение карт и логику их взаимодействия, выполняют:

1. CardsAdapter — универсальный адаптер (наследник RecyclerView.Adapter), параметризуемый под разные контексты (рука, поле, колода, диалог) через настройки layoutRes, showPower и powerCalculator.
2. CardPreviewDialogFragment — диалоговый фрагмент для просмотра и выбора карт.
3. ModeSelectAdapter — адаптер для отображения карточек режимов игры.

Такая организация позволяет избежать дублирования кода разметки и единообразно отображать карты во всех частях приложения.

7.2.2 Соблюдение инкапсуляции

Взаимодействие интерфейса с игровым ядром построено на принципе инкапсуляции: активности не изменяют состояние напрямую, а вызывают публичные методы менеджеров.

Пример: взаимодействия пользователя с ядром. Когда игрок кликает по карте в руке, активность вызывает метод playCard() ядра. Ядро обрабатывает действие: удаляет карту из руки, добавляет в ряд, применяет способность. Активность не знает, какие именно изменения происходят — она только получает команду и перерисовывает интерфейс после её выполнения.

Пример обратной связи. Когда ядро завершает раунд, оно сообщает об этом активности через колбэк. Активность обновляет интерфейс: показывает оверлей с результатом, очищает поле, готовит следующий раунд.

Такой подход исключает ситуацию, при которой интерфейс и логика могут «разойтись» — данные всегда изменяются в одном месте.

7.2.3 Сценарии ввода и вывода данных

Ввод данных. Пользователь взаимодействует с приложением через жесты: одинарный клик, двойной клик, долгое нажатие. Эти жесты преобразуются в вызовы методов ядра:

1. двойной клик по карте в руке — розыгрыш карты;
2. одинарный клик по кнопке «Пас» — объявление паса;
3. долгое нажатие на карту — открытие диалога просмотра карт;
4. одинарный клик по карте в диалоге просмотра карт — отображение описания способности карты;
5. клик по кнопке сброса — открытие списка карт в сбросе.

Вывод данных. Результаты работы ядра отображаются в интерфейсе:

6. количество очков каждого ряда и суммарный счёт;
7. состав карт на поле и в руке;
8. активные погодные эффекты;
9. счётчик карт в колоде;
10. всплывающие сообщения о ходе противника, объявлении паса и другие.

Все изменения интерфейса происходят через перерисовку элементов после завершения операции ядра.

7.3 Роль: Специалист по безопасности и валидации данных (Security & Data Intergrity)

7.3.1 Приватные и защищённые атрибуты

Приватные атрибуты — это поля класса, доступные только внутри самого класса. Они объявляются с модификатором доступа private и не могут быть изменены извне. В проекте все поля состояния игровой логики объявлены как приватные, что видно в примерах кода выше.

Такая организация исключает случайное изменение состояния из других классов и гарантирует, что все изменения происходят через контролируемые точки входа.

Защищённые атрибуты — это поля, доступные внутри класса и его наследников. В проекте этот модификатор применяется ограниченно, поскольку наследование используется редко.

7.3.2 Сеттеры и геттеры как фильтры

Сеттер — это метод, устанавливающий значение поля. Геттер — метод, возвращающий значение. В проекте сеттеры и геттеры инкапсулируют логику работы с данными. Пример сеттера — установка громкости в SoundManager: он принимает целое число и выполняет нормализацию значения (volume / 100.0f) для передачи в MediaPlayer. Пример геттера — загрузка настроек, где при отсутствии сохраненного значения возвращается безопасный дефолт (100), что исключает пустой или некорректный старт приложения.

Пример сеттера — установка громкости, реализцию которой можно увидеть на рисунке 8.

```
2 Usages
fun setVolume(volume: Int) {
    soundVolume = volume / 100.0f
    mediaPlayers.values.forEach { it.setVolume( leftVolume = soundVolume,  rightVolume = soundVolume) }
}
```

Рисунок 8. Пример кода сеттера

Пример геттера — загрузка настроек, рисунок 9.

```
val savedVolume = prefs.getInt( p0 = "music_volume",  p1 = 100)
```

Рисунок 9. Пример кода геттера

Если значение отсутствует, возвращается значение по умолчанию.

7.3.3 Проверки типов и значений

Проверка типов — операция, при которой убеждаются, что значение принадлежит ожидаемому типу. В Kotlin проверка типов выполняется средствами языка: несоответствие типов выявляется на этапе компиляции.

Проверка значений выполняется во время выполнения. В проекте применяются следующие проверки:

— ограничение значений громкости элементами интерфейса SeekBar (диапазон от 0 до 100) перед передачей в SoundManager;

— проверка на null при загрузке сохранений через оператор ?: ;

— проверка размера коллекции перед розыгрышем карты;

— проверка условий игровых действий: например, лидер может быть активирован только один раз за партию.

7.3.4 Исключения и обработка ошибок

Исключение — это объект, сигнализирующий о возникновении ошибки. Исключения перехватываются конструкцией try-catch, которая позволяет обработать ошибку и продолжить работу. Несколько примеров приведены ниже

Пример — загрузка описаний способностей, рисунок 10

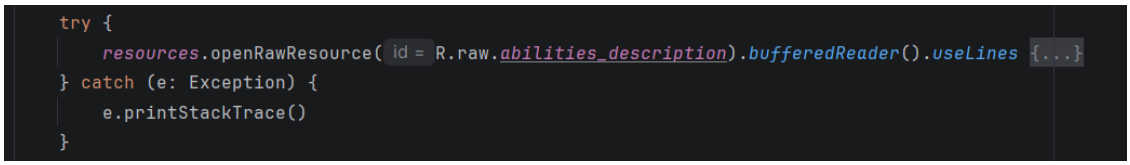


Рисунок 10. Пример кода конструкции try-catch

Если файл недоступен или повреждён, приложение продолжает работу без описаний, вместо аварийного завершения.

Пример — освобождение ресурсов медиаплеера, рисунок 11.

```
private fun releaseMusic() {
    try {
        mediaPlayer?.stop()
    } catch (e: Exception) {
    }
    try {
        mediaPlayer?.release()
    } catch (e: Exception) {
    }
    mediaPlayer = null
}
```

Рисунок 11. Пример кода освобождения ресурсов мультиплеера

Это исключает падение приложения при освобождении ресурсов, которые уже могли быть освобождены ранее.

7.3.5 Модуль валидации

Модуль валидации — совокупность методов, отвечающих за проверку корректности данных. Роль модуля валидации в проекте распределена между несколькими классами.

В классе-репозитории DeckRepository реализована проверка наличия сохранённых данных. Если ключ отсутствует, метод возвращает пустую колоду или null, а вызывающий код (например, ModeSelectActivity) безопасно подставляет лидера по умолчанию (CardsData.leaderCards[0]).

В классе BattleActivity реализованы проверки игровых действий: возможность розыгрыша карты, активации лидера, объявления паса.

В классе SoundManager реализована валидация громкости и безопасное освобождение ресурсов.

Пример кода валидации представлен на рисунке 12

```
fun loadLeader(): Cards? {
    val json = prefs.getString( p0 = "player_leader",  p1 = null) ?: return null
    return gson.fromJson(json,  classOfT = Cards::class.java)
}
```

Рисунок 12. Пример кода валидации

Если сохранённые данные отсутствуют (ключ не найден), метод возвращает null, и вызывающий код подставляет лидера по умолчанию. Для предотвращения неконтролируемых ситуаций все операции работы с хранилищем инкапсулированы в DeckRepository.

7.3.6 Законы и персональных данных

Требования Федерального закона № 152-ФЗ «О персональных данных» к проекту применяются минимально, поскольку приложение не собирает и не обрабатывает персональные данные. Все данные хранятся локально и не передаются на серверы. Регламент GDPR неактуален, поскольку приложение ориентировано на российский рынок. Перед публикацией в магазинах Google Play или RuStore потребуется предоставить декларацию об отсутствии сбора персональных данных.

7.4 Заключение

В рамках работы над проектом автор совместил три роли: архитектора ядра, разработчика интерфейса и специалиста по безопасности и валидации данных. Основной ролью являлась роль архитектора ядра, в рамках которой была спроектирована и реализована модульная структура игрового ядра с разделением на менеджеры и состояние боя. В роли разработчика интерфейса реализованы экраны главного меню, редактора колоды и сражения с единым адаптером отображения карт. В роли специалиста по безопасности и валидации данных реализованы механизмы обработки ошибок, ограничения значений и инкапсуляции хранилища.

Совмещение ролей позволило обеспечить согласованность решений на всех уровнях системы: архитектурные принципы, реализованные в ядре, находят отражение в интерфейсе, а механизмы безопасности встроены в обе части приложения.